

pgAdmin 4

Object Explorer

- aula
- bd_escola
- bdabp
- bdaula
- biblioteca**
 - Casts
 - Catalogs
 - Event Triggers
 - Extensions
 - Foreign Data Wrappers
 - Languages
 - Publications
 - Schemas(1)
 - public
 - Aggregates
 - Collations
 - Domains
 - FTS Configurations
 - FTS Dictionaries
 - FTS Parsers
 - FTS Templates
 - Foreign Tables
 - Functions
 - Materialized Views
 - Operators
 - Procedures
 - Sequences
 - Tables(2)
 - autor
 - livro

Dashboard | Properties | SQL | Statistics | Dependencies | Dependents | Processes | biblioteca/postgres@PostgreSQL 17*

Query History

```

45 -- EXERCÍCIO 1 [cite: 925, 926, 927]
46 -- Usando subquery scalar no SELECT, listar cada autor e quantos livros publicou
47 -- (COUNT) e a média de páginas. Reescrever o mesmo usando CTE e comparar legibilidade.
48 -----
49
50 -- Parte 1: Subquery Scalar no SELECT [cite: 877-882]
51 SELECT a.nome,
52        (SELECT COUNT(*) FROM livro l WHERE l.id_autor = a.id_autor) AS total_livros,
53        ROUND((SELECT AVG(l2.num_paginas) FROM livro l2 WHERE l2.id_autor = a.id_autor), 2) AS media_paginas
54 FROM autor a
55 ORDER BY a.nome;
56
57 -- Parte 2: Reescrevendo usando CTE [cite: 883-893]
58 WITH livros_por_autor AS (
59     SELECT id_autor, COUNT(*) AS total_livros, AVG(num_paginas) AS media_paginas
60 FROM livro
61 GROUP BY id_autor
62 )
63 SELECT a.nome,
64        COALESCE(lpa.total_livros, 0) AS total_livros,
65        ROUND(lpa.media_paginas::numeric, 2) AS media_paginas
66 FROM autor a
67 LEFT JOIN livros_por_autor lpa ON lpa.id_autor = a.id_autor
68 ORDER BY a.nome;
69
70 /* Comparação de legibilidade exigida no exercício:

```

Data Output Messages Notifications

Showing rows: 1 to 4 Page No: 1 of 1

	nome	total_livros	media_paginas
	character varying (100)	bigint	numeric
1	Clarice Lispector	1	88.00
2	J. R. R. Tolkien	2	952.00
3	J.K. Rowling	0	[null]
4	Machado de Assis	1	208.00

Successfully run. Total query runtime: 138 msec. 4 rows affected.

Total rows: 4 Query complete 00:00:00.146

pgAdmin 4

Object Explorer

- aula
- bd_escola
- bdabp
- bdaula
- biblioteca**
 - Casts
 - Catalogs
 - Event Triggers
 - Extensions
 - Foreign Data Wrappers
 - Languages
 - Publications
 - Schemas(1)
 - public
 - Aggregates
 - Collations
 - Domains
 - FTS Configurations
 - FTS Dictionaries
 - FTS Parsers
 - FTS Templates
 - Foreign Tables
 - Functions
 - Materialized Views
 - Operators
 - Procedures
 - Sequences
 - Tables(2)
 - autor
 - livro

Dashboard | Properties | SQL | Statistics | Dependencies | Dependents | Processes | biblioteca/postgres@PostgreSQL 17*

Query History

```

56
57 -- Parte 2: Reescrevendo usando CTE [cite: 883-893]
58 WITH livros_por_autor AS (
59     SELECT id_autor, COUNT(*) AS total_livros, AVG(num_paginas) AS media_paginas
60 FROM livro
61 GROUP BY id_autor
62 )
63 SELECT a.nome,
64        COALESCE(lpa.total_livros, 0) AS total_livros,
65        ROUND(lpa.media_paginas::numeric, 2) AS media_paginas
66 FROM autor a
67 LEFT JOIN livros_por_autor lpa ON lpa.id_autor = a.id_autor
68 ORDER BY a.nome;
69
70 /* Comparação de legibilidade exigida no exercício:

```

Data Output Messages Notifications

Showing rows: 1 to 4 Page No: 1 of 1

	nome	total_livros	media_paginas
	character varying (100)	bigint	numeric
1	Clarice Lispector	1	88.00
2	J. R. R. Tolkien	2	952.00
3	J.K. Rowling	0	[null]
4	Machado de Assis	1	208.00

Successfully run. Total query runtime: 83 msec. 4 rows affected.

Total rows: 4 Query complete 00:00:00.083

pgAdmin 4

File Object Tools Edit View Window Help

Object Explorer

- aula
- bd_escola
- bdabp
- bdaula
- biblioteca**
 - Casts
 - Catalogs
 - Event Triggers
 - Extensions
 - Foreign Data Wrappers
 - Languages
 - Publications
 - Schemas(1)
 - public
 - Aggregates
 - Collations
 - Domains
 - FTS Configurations
 - FTS Dictionaries
 - FTS Parsers
 - FTS Templates
 - Foreign Tables
 - Functions
 - Materialized Views
 - Operators
 - Procedures
 - 1.3 Sequences
 - Tables(2)
 - autor
 - livro

Dashboard Properties SQL Statistics Dependencies Dependents Processes biblioteca/postgres@PostgreSQL 17*

Query Query History

```
-- Depois compare com média geral (subquery) e retorne autores acima da média.

WITH paginas_por_autor AS (
  SELECT a.id_autor, a.nome AS autor, SUM(l.num_paginas) AS total_paginas
  FROM autor a
  JOIN livro l ON a.id_autor = l.id_autor
  GROUP BY a.id_autor, a.nome
)
SELECT p.autor, p.total_paginas
FROM paginas_por_autor p
WHERE p.total_paginas > (
  SELECT AVG(total_paginas) FROM paginas_por_autor
);
```

Data Output Messages Notifications

	autor	total_paginas
	character varying (100)	bigint
1	J. R. R. Tolkien	1904

Showing rows: 1 to 1 Page No: 1 of 1

Successfully run. Total query runtime: 111 msec. 1 rows affected.

Total rows: 1 Query complete 00:00:00.111

pgAdmin 4

File Object Tools Edit View Window Help

Object Explorer

- aula
- bd_escola
- bdabp
- bdaula
- biblioteca**
 - Casts
 - Catalogs
 - Event Triggers
 - Extensions
 - Foreign Data Wrappers
 - Languages
 - Publications
 - Schemas(1)
 - public
 - Aggregates
 - Collations
 - Domains
 - FTS Configurations
 - FTS Dictionaries
 - FTS Parsers
 - FTS Templates
 - Foreign Tables
 - Functions
 - Materialized Views
 - Operators
 - Procedures
 - 1.3 Sequences
 - Tables(2)
 - autor
 - livro

Dashboard Properties SQL Statistics Dependencies Dependents Processes biblioteca/postgres@PostgreSQL 17*

Query Query History

```
SELECT t.autor, t.total_paginas
FROM (
  SELECT autor.id_autor, autor.nome AS autor, SUM(livro.num_paginas) AS total_paginas
  FROM autor
  INNER JOIN livro ON autor.id_autor = livro.id_autor
  GROUP BY autor.id_autor, autor.nome
) t
WHERE t.total_paginas > (
  SELECT AVG(sub.total_paginas)
  FROM (
    SELECT SUM(livro.num_paginas) AS total_paginas
    FROM livro
    GROUP BY livro.id_autor
  ) sub
);
```

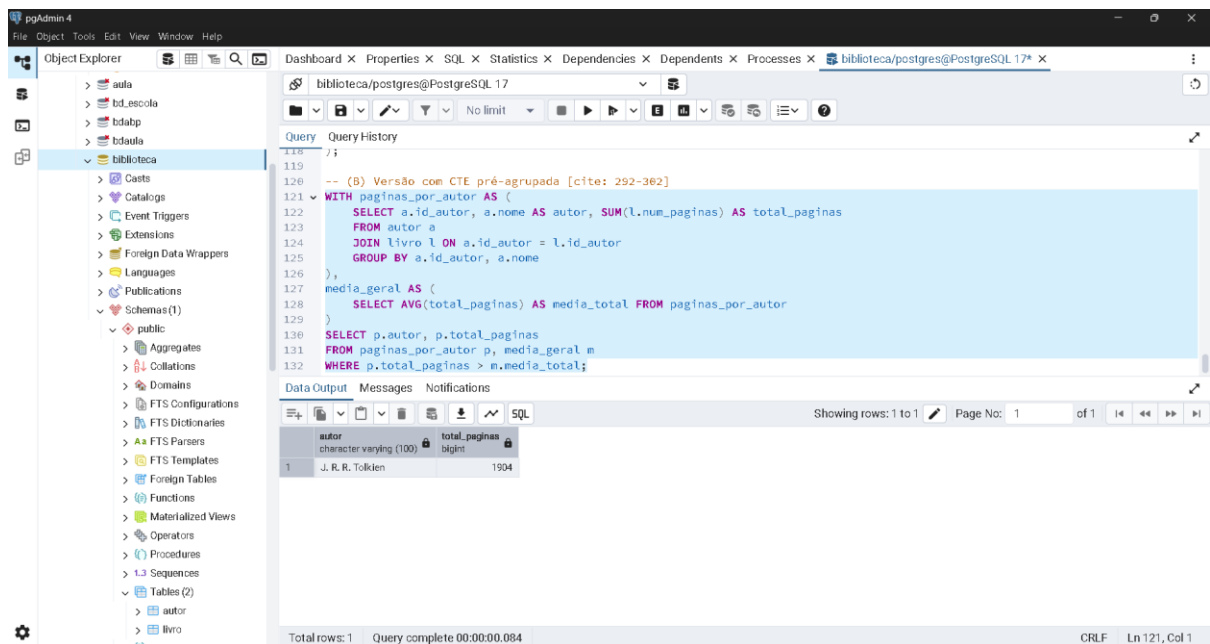
Data Output Messages Notifications

	autor	total_paginas
	character varying (100)	bigint
1	J. R. R. Tolkien	1904

Showing rows: 1 to 1 Page No: 1 of 1

Successfully run. Total query runtime: 76 msec. 1 rows affected.

Total rows: 1 Query complete 00:00:00.076



--

=====

===== -- ARQUIVO: schema.sql -- DISCIPLINA: Banco de Dados Relacional - Fatec Jacareí -- ATIVIDADE: Atividade Prática - Aula 11 (Tema - GROUP BY com JOINS e Relatórios Avançados) --

=====

-- 0. ESTRUTURA DO BANCO DE DADOS (DDL) E INSERÇÃO DE DADOS (DML)

-- Removendo tabelas se já existirem para evitar erros na execução DROP TABLE IF EXISTS livro; DROP TABLE IF EXISTS autor;

-- Criação da tabela autor CREATE TABLE autor (id_autor INTEGER PRIMARY KEY, nome VARCHAR(100) NOT NULL);

-- Criação da tabela livro CREATE TABLE livro (id_livro INTEGER PRIMARY KEY, titulo VARCHAR(150) NOT NULL, ano_publicacao INTEGER, id_autor INTEGER REFERENCES autor(id_autor), id_editora INTEGER, num_paginas INTEGER);

-- Inserção de dados dos autores INSERT INTO autor (id_autor, nome) VALUES (1, 'J. R. R. Tolkien'), (2, 'Machado de Assis'), (3, 'Clarice Lispector'), (4, 'J.K. Rowling');

-- Inserção de dados dos livros INSERT INTO livro (id_livro, titulo, ano_publicacao, id_autor, id_editora, num_paginas) VALUES (1, 'O Senhor dos Anéis', 1954, 1, NULL,

1568), (2, 'Dom Casmurro', 1899, 2, 2, 208), (3, 'A Hora da Estrela', 1977, 3, 3, 88), (4, 'O Hobbit', 1937, 1, 1, 336);

-- EXERCÍCIO 1 [cite: 925, 926, 927] -- Usando subquery scalar no SELECT, listar cada autor e quantos livros publicou -- (COUNT) e a média de páginas. Reescrever o mesmo usando CTE e comparar legibilidade.

-- Parte 1: Subquery Scalar no SELECT [cite: 877-882] SELECT a.nome, (SELECT COUNT(*) FROM livro l WHERE l.id_autor = a.id_autor) AS total_livros, ROUND((SELECT AVG(l2.num_paginas) FROM livro l2 WHERE l2.id_autor = a.id_autor), 2) AS media_paginas FROM autor a ORDER BY a.nome;

-- Parte 2: Reescrevendo usando CTE [cite: 883-893] WITH livros_por_autor AS (SELECT id_autor, COUNT(*) AS total_livros, AVG(num_paginas) AS media_paginas FROM livro GROUP BY id_autor) SELECT a.nome, COALESCE(lpa.total_livros, 0) AS total_livros, ROUND(lpa.media_paginas::numeric, 2) AS media_paginas FROM autor a LEFT JOIN livros_por_autor lpa ON lpa.id_autor = a.id_autor ORDER BY a.nome;

/* Comparação de legibilidade exigida no exercício: O uso da CTE (Common Table Expression) torna a consulta muito mais legível e organizada, pois a agregação (COUNT e AVG) é processada apenas uma vez na tabela base[cite: 326]. No formato de subquery scalar, a lógica de filtragem (WHERE) precisa ser repetida várias vezes para cada nova coluna calculada [cite: 120-125], o que polui o código. */

-- EXERCÍCIO 2 [cite: 928, 929, 934] -- Listar autores cuja soma de páginas ultrapassa a média geral. -- Primeiro calcule paginas_por_autor (CTE ou derived table). -- Depois compare com média geral (subquery) e retorne autores acima da média.

WITH paginas_por_autor AS (SELECT a.id_autor, a.nome AS autor, SUM(l.num_paginas) AS total_paginas FROM autor a JOIN livro l ON a.id_autor = l.id_autor GROUP BY a.id_autor, a.nome) SELECT p.autor, p.total_paginas FROM paginas_por_autor p WHERE p.total_paginas > (SELECT AVG(total_paginas) FROM paginas_por_autor);

-- EXERCÍCIO 3 [cite: 935, 936, 937] -- Escreva duas versões da mesma consulta: (A) subconsulta correlacionada; -- (B) CTE pré-agrupada. -- (Utilizando a listagem de páginas acima da média para demonstrar).

```
-- (A) Versão com Subconsulta no FROM (Derived Table / Correlacionada) [cite: 257-270] SELECT t.autor, t.total_paginas FROM ( SELECT autor.id_autor, autor.nome AS autor, SUM(livro.num_paginas) AS total_paginas FROM autor INNER JOIN livro ON autor.id_autor = livro.id_autor GROUP BY autor.id_autor, autor.nome ) t WHERE t.total_paginas > ( SELECT AVG(sub.total_paginas) FROM ( SELECT SUM(livro.num_paginas) AS total_paginas FROM livro GROUP BY livro.id_autor ) sub );
```

```
-- (B) Versão com CTE pré-agrupada [cite: 292-302] WITH paginas_por_autor AS ( SELECT a.id_autor, a.nome AS autor, SUM(l.num_paginas) AS total_paginas FROM autor a JOIN livro l ON a.id_autor = l.id_autor GROUP BY a.id_autor, a.nome ), media_geral AS ( SELECT AVG(total_paginas) AS media_total FROM paginas_por_autor ) SELECT p.autor, p.total_paginas FROM paginas_por_autor p, media_geral m WHERE p.total_paginas > m.media_total;
```